

SC102 L15：背包与物品系统——详细学习笔记

涵盖：背包系统架构 | 物品数据结构 | 物品管理器 | 背包 UI | 拖拽交互

1 背包系统概述

背包系统（Inventory System）是 RPG 和许多游戏中核心的 Gameplay 系统之一。一个好的背包系统需要处理：物品数据建模、物品的增删改查、UI 显示与交互（拖拽、使用、丢弃）、与存档系统的集成。

2 物品数据结构设计

核心：使用 ScriptableObject 定义物品模板数据（物品名、图标、描述、堆叠数量上限等），运行时每个背包槽位存储物品 ID+ 数量。

```
[CreateAssetMenu(fileName = "NewItem", menuName = "Inventory/Item")]
public class ItemData : ScriptableObject {
    public string itemName;           // 物品名称
    public Sprite icon;               // 物品图标
    [TextArea] public string description; // 物品描述
    public int maxStackSize = 99;     // 最大堆叠数
    public ItemType type;             // 物品类型枚举
}

public enum ItemType { Consumable, Equipment, Quest, Material }

[System.Serializable]
public class InventorySlot {
    public ItemData item;             // 物品数据引用
    public int quantity;              // 当前数量
    public bool IsEmpty => item == null || quantity <= 0;
}
```

3 物品管理器 (Inventory Manager)

背包管理器负责数据的增删改查。可以使用单例模式方便全局访问：

```
public class InventoryManager : MonoBehaviour {
    public static InventoryManager Instance { get; private set; }
    public List<InventorySlot> slots = new List<InventorySlot>(30);
    public int maxSlots = 30;

    void Awake() { Instance = this; }

    public bool AddItem(ItemData item, int quantity = 1) {
        // 1. 先尝试堆叠到已有的同类物品
        foreach (var slot in slots) {
            if (slot.item == item && slot.quantity < item.maxStackSize) {
                int canAdd = Mathf.Min(quantity,
                    item.maxStackSize - slot.quantity);
                slot.quantity += canAdd;
                quantity -= canAdd;
                if (quantity <= 0) return true;
            }
        }
        // 2. 有空位则放入新物品
        while (quantity > 0) {
            int emptyIndex = slots.FindIndex(s => s.IsEmpty);
            if (emptyIndex < 0 || slots.Count >= maxSlots)
                return false; // 背包已满
            int toAdd = Mathf.Min(quantity, item.maxStackSize);
            slots[emptyIndex].item = item;
            slots[emptyIndex].quantity = toAdd;
            quantity -= toAdd;
        }
        return true;
    }
}
```

4 背包 UI

使用 Grid Layout Group + Scroll View 构建背包网格。每个格子是一个 UI 预制件（显示图标 + 数量）。

核心交互： 点击物品 → 显示详情；拖拽物品 → 交换/移动位置；右键物品 → 使用/丢弃。

5 课后实践要点

完成背包系统需要整合：ScriptableObject 物品模板 → InventoryManager 数据层 → 背包 UI → 拖拽交互（使用 L12 的拖动系统）→ 物品使用效果（与战斗/对话系统联动）。

6 总结

背包系统的核心分层：

1. **数据层**：ItemData (ScriptableObject) + InventorySlot
2. **逻辑层**：InventoryManager（增删改查、堆叠逻辑）
3. **表现层**：Grid-based UI + 拖拽交互
4. **持久层**：存档/读档（序列化 InventorySlot 列表）